laz-viewer

Technical document: architecture, workflows and operations

Repository	github.com/sarahmehosay/laz-viewer
Live viewer	https://sarahmehosay.github.io/laz-viewer/
Reflects	Code as of 30 October 2026
Audience	Developers who build, run, extend or deploy the project

1. Overview

laz-viewer displays LAS and LAZ point clouds on a 3D CesiumJS globe in the browser. A file is decoded locally in the browser tab, decimated to an interactive size and drawn with its RGB colours, or coloured by height when it has none.

An optional Python backend extracts the ground surface from a point cloud and returns ground and non-ground points as separate LAS files. The backend runs only in the Docker setup: GitHub Pages hosts static files, so the deployed site is the viewer alone.

Main capabilities

- LAS/LAZ 1.0–1.4, point data formats 0–10; LAZ decompressed with laz-rs compiled to WebAssembly.
- Automatic decimation to at most 800,000 rendered points; files up to 1.5 GiB.
- RGB colour when present; height colour ramp when absent or when every point has the same RGB.
- Placement on the globe at a user-adjustable anchor (latitude, longitude, height).
- Ground extraction with a progressive morphological filter (backend, optional).

Components at a glance

Part	Technology	Responsibility
frontend/	Vue 3, Vite 8, Tailwind CSS 4, CesiumJS, @loaders.gl/las	File input, LAS/LAZ decoding, 3D rendering, ground extraction UI
backend/	Python 3.12, FastAPI, laspy + lazrs, numpy, scipy	Upload handling, background jobs, ground filter, LAS output
Hosting	GitHub Pages (static) or Docker Compose (nginx + backend)	Serve the built frontend; proxy /api to the backend in Docker
CI/CD	GitHub Actions	Build the frontend and publish it to GitHub Pages

2. Architecture

The frontend and backend are independent folders with their own dependencies. They communicate only over HTTP under `/api`, so either can be deployed on its own.

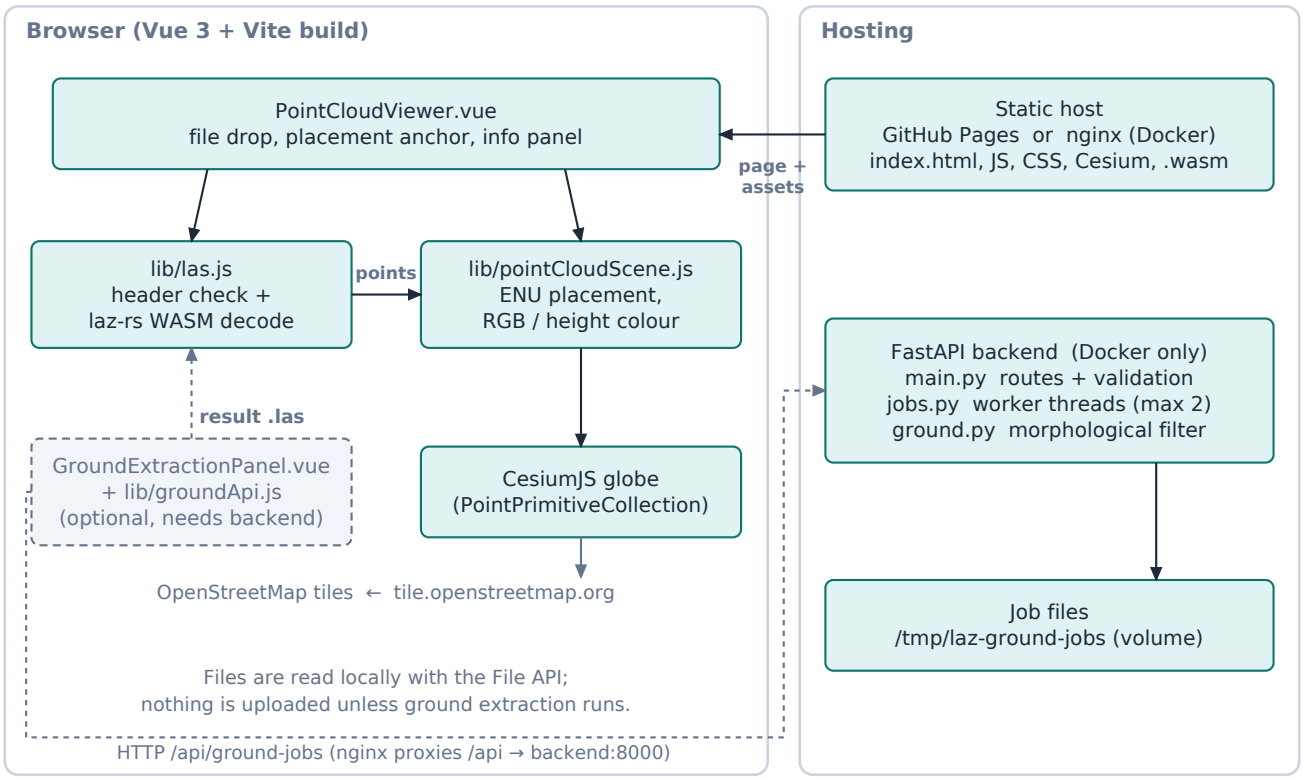


Figure 1. Components and data flow. The dashed parts are only active when a backend is reachable.

Repository layout

Path	Contents
`frontend/src/components/PointCloudViewer.vue`	Main UI: Cesium viewer, file drop zone, info and placement panels
`frontend/src/components/GroundExtractionPanel.vue`	Ground extraction parameters, job progress, result views and downloads
`frontend/src/lib/las.js`	Header inspection, decoding via LAZRsLoader, decimation, blank-RGB detection
`frontend/src/lib/pointCloudScene.js`	Places points on the globe and colours them; camera framing
`frontend/src/lib/groundApi.js`	HTTP client for the ground-job API
`backend/app/main.py`	FastAPI routes, upload and parameter validation, CORS
`backend/app/jobs.py`	In-memory job store, worker threads, writing result files
`backend/app/ground.py`	Progressive morphological ground filter (pure numpy/scipy)
`.github/workflows/deploy-pages.yml`	Build and publish the frontend to GitHub Pages
`docker-compose.yml`	nginx frontend on port 8080 plus the backend and a job-file volume

3. Workflow: loading and rendering a file

Everything in this workflow happens in the browser. The header is checked before the whole file is read, so invalid or unsupported files fail fast and the decimation factor is known before decoding starts.

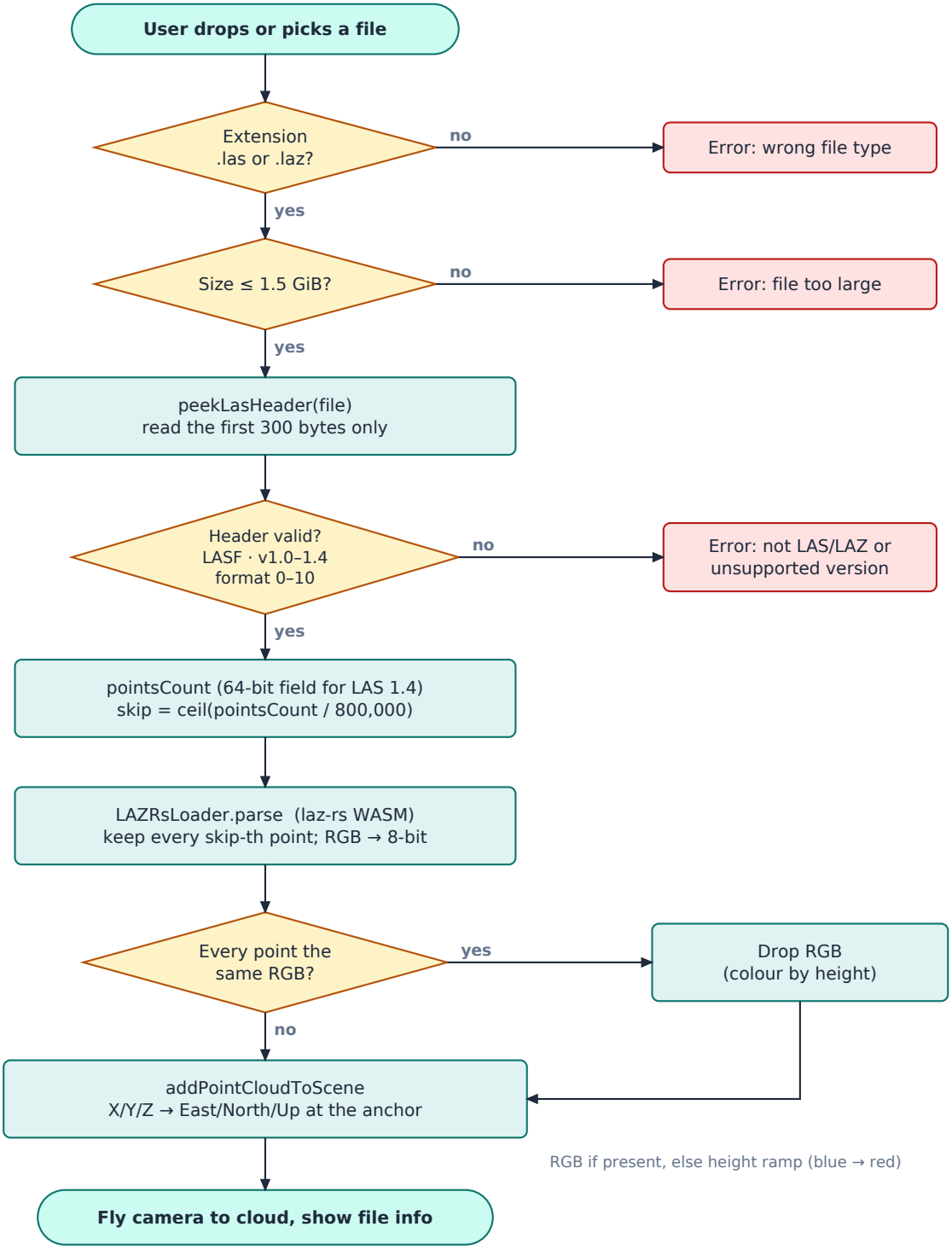


Figure 2. File loading workflow in PointCloudViewer.vue and lib/las.js.

3.1 Supported files

Item	Support	Notes
LAS versions	1.0 – 1.4	1.4 point count read from the 64-bit header field (offset 247)
Point formats	0 – 10	RGB used for 2, 3, 5, 7, 8, 10; NIR, waveforms and extra bytes are ignored
Compression	LAS and LAZ	LAZ via laz-rs (WASM, ~49 kB gzipped), served next to the app's JS
File size	≤ 1.5 GiB	The whole file is held in memory in the browser tab
Rendered points	≤ 800,000	skip = ceil(points / 800,000); every skip-th point is kept

3.2 Colour

16-bit RGB is scaled to 8 bits (values above 255 trigger scaling). Some scanners write the RGB fields without a camera, so every point is (0, 0, 0) or (255, 255, 255). When all loaded points share one colour, the viewer discards it, colours by height and says so in the info panel. Height colouring maps the lowest point to blue and the highest to red (HSL hue 0.66 → 0).

3.3 Placement on the globe

LAS files carry no coordinate reference system the viewer can rely on. Instead of guessing a projection, the cloud's bounding-box centre is placed at the anchor point, and each point's local X, Y and Z become East, North and Up offsets in a local tangent plane (`Transforms.eastNorthUpToFixedFrame`). Shape and scale are exact; the real-world position is only as accurate as the anchor. The default anchor is Tokyo Tower (35.6586, 139.7454); “Reload at new anchor” re-parses the file at a new position.

4. Workflow: ground extraction

Ground extraction runs as an asynchronous job so large files do not block the HTTP request. The panel polls for progress and loads the ground result into the viewer when the job finishes. The panel is optional: it is the only part of the frontend that needs the backend.

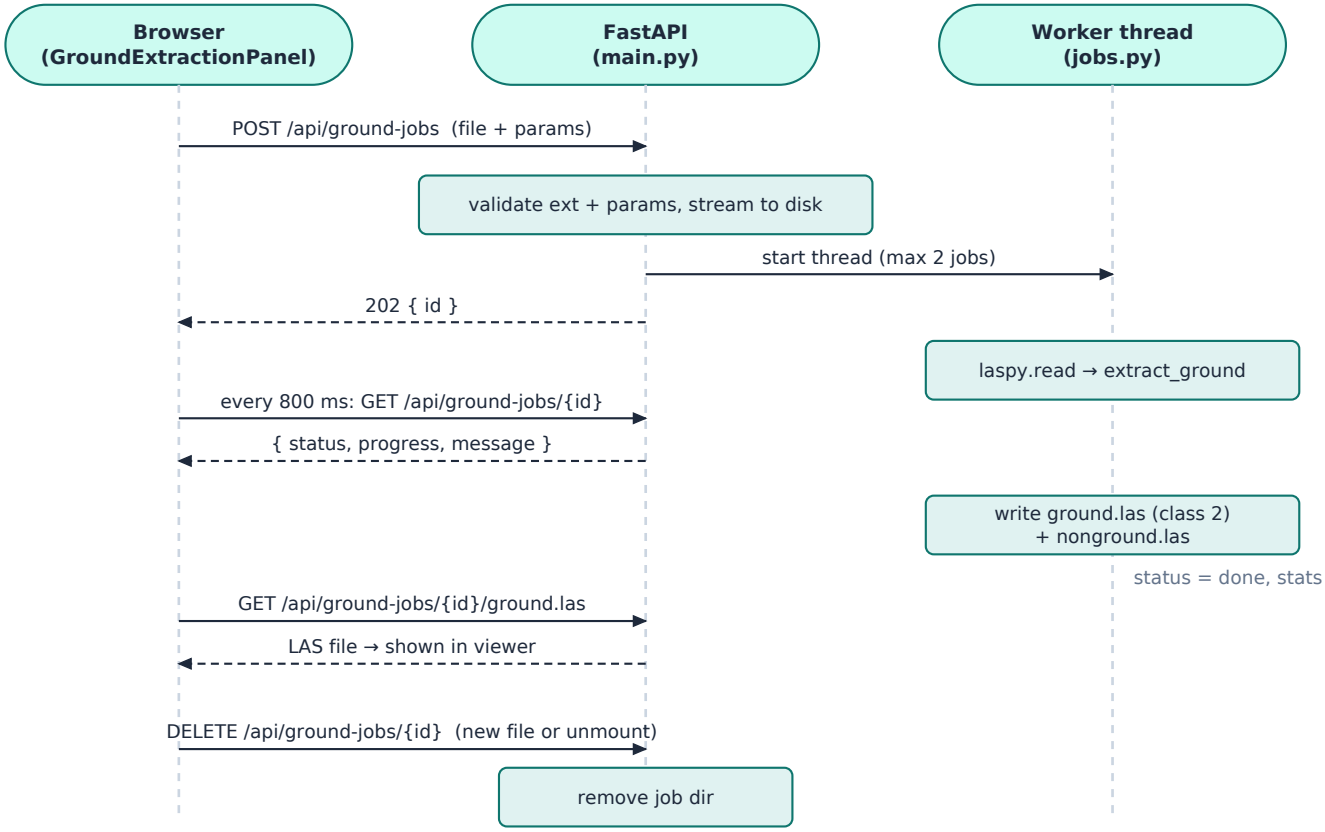
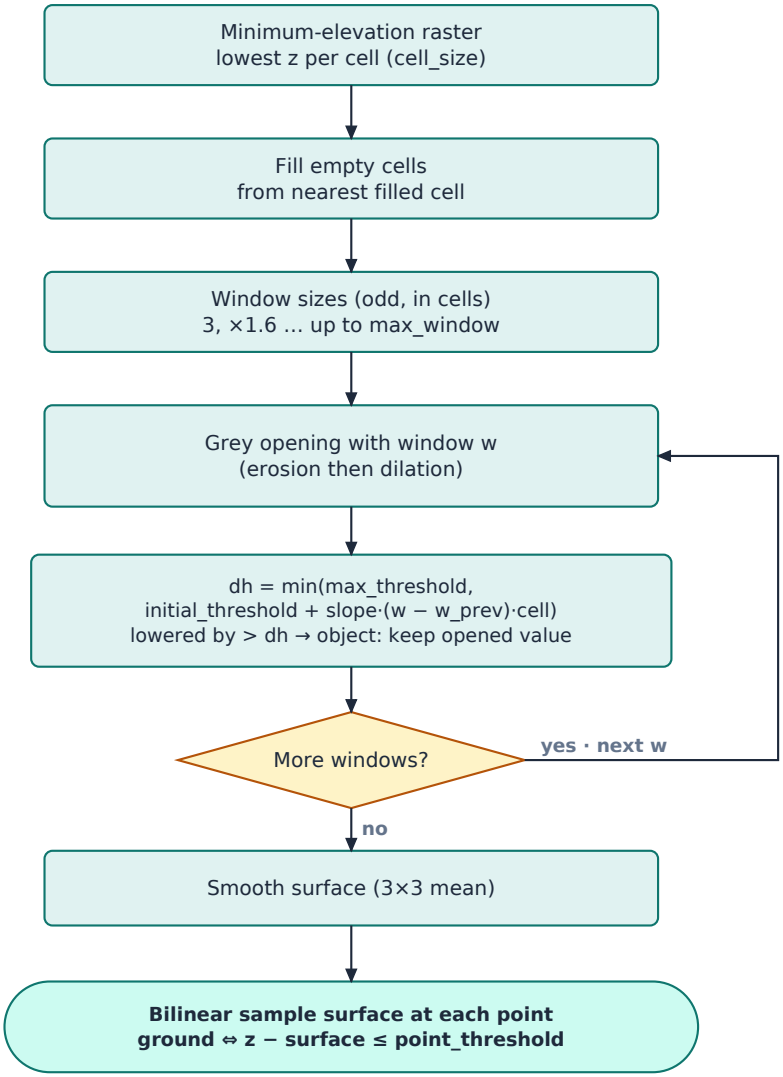


Figure 3. Ground job sequence between the panel, the API and a worker thread.

Job states are `queued` → `running` → `done` or `error`. At most two jobs run at once (the filter is memory-bound). Jobs live in memory, so restarting the backend forgets them; their files stay in the job volume until deleted.

4.1 Ground filter algorithm

The filter works on a raster of minimum elevations rather than on individual points, which keeps it fast (seconds for tens of millions of points). Larger windows remove larger objects; the height threshold grows with the window so sloped terrain is not mistaken for objects.



Zhang et al. (2003), progressive morphological filter

Figure 4. Progressive morphological filter in backend/app/ground.py.

Parameters

Parameter	Default	Allowed	Effect
`cell_size`	1.0 m	> 0.05, ≤ 50	Raster resolution. Larger is faster but coarser
`max_window`	30 m	> 1, ≤ 500	Largest object (e.g. building) to remove
`slope`	0.15	0 – 2	Terrain rise/run tolerated; raise for steep ground
`initial_threshold`	0.3 m	0 – 5	Height change that marks an object at the smallest window
`max_threshold`	3.0 m	> 0, ≤ 50	Upper limit of the height threshold
`point_threshold`	0.3 m	0 – 5	Max height above the surface for a ground point

Rasters larger than 100 million cells are rejected with a request to increase `cell_size`.

4.2 HTTP API

Method	Path	Purpose	Responses
GET	`/api/health`	Liveness check	200 {ok: true}
POST	`/api/ground-jobs`	Multipart upload: file + parameters	202 {id}; 400 bad extension; 413 > 2 GiB; 422 bad parameter
GET	`/api/ground-jobs/{id}`	Status, progress (0–1), message, error, stats	200; 404 unknown job
GET	`/api/ground-jobs/{id}/ground.las`	Ground points, classification 2	200; 404 if not done
GET	`/api/ground-jobs/{id}/nonground.las`	All other points	200; 404 if not done
DELETE	`/api/ground-jobs/{id}`	Remove job and its files	204; 404 unknown job

Results keep the input's LAS version and point format. Interactive API docs are at `/docs` when the backend is reached directly on port 8000.

5. Running and deploying

5.1 Local development

```
# Viewer only (Node 20.19+ or 22+)
cd frontend && npm install && npm run dev        # http://localhost:5173

# Backend for ground extraction (Python 3.10+)
cd backend
python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
.venv/bin/uvicorn app.main:app --reload          # http://localhost:8000

# Everything in Docker
docker compose up --build                        # http://localhost:8080
```

In development, Vite proxies `/api` to `localhost:8000`. In Docker, nginx serves the built frontend and proxies `/api` to the backend container, so no CORS setup is needed.

5.2 GitHub Pages deployment

Every push to `main` that touches `frontend/` or the workflow file builds and publishes the frontend; it can also be started from the Actions tab. Pages must be set to “Source: GitHub Actions”.

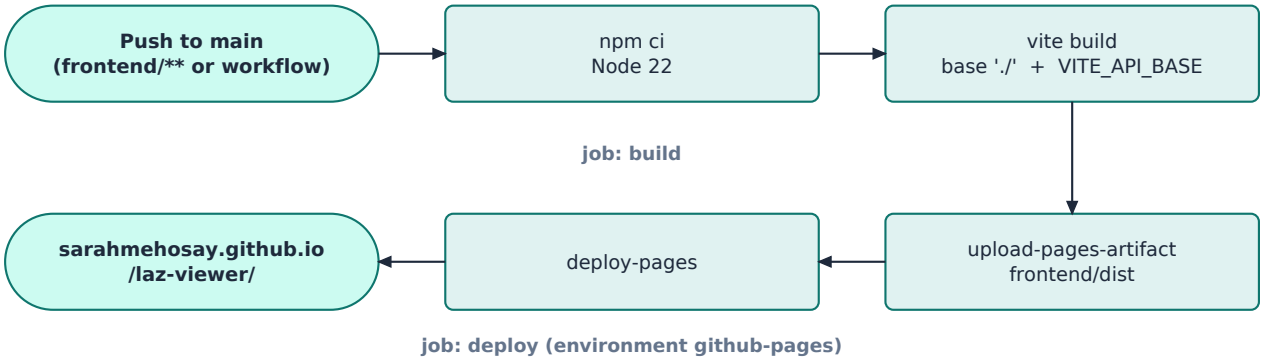


Figure 5. .github/workflows/deploy-pages.yml.

The build uses a relative base path (`base: './'`). An absolute sub-path would make vite-plugin-cesium copy Cesium's assets into the wrong folder, and relative paths work on Pages, in Docker and at any other sub-path.

5.3 Configuration

Variable	Where	Purpose
`VITE_API_TARGET`	Frontend dev server	Backend URL for the /api proxy (default http://localhost:8000)
`VITE_API_BASE`	Frontend build; repo variable in Actions	Public backend URL prefixed to API calls (empty = same origin)
`CORS_ORIGINS`	Backend environment	Extra allowed origins, comma-separated, e.g. https://sarahmehosay.github.io

6. Testing

Suite	Command	Covers
Frontend (Vitest, Vue Test Utils, happy-dom)	`cd frontend && npm test`	Header parsing, LAS 1.4 and LAZ decoding, decimation, colour scaling and blank-RGB detection, API client errors, scene placement and colouring, ground panel job flow
Backend (pytest)	`cd backend && .venv/bin/python -m pytest`	Filter accuracy on synthetic terrain, window sizes, progress, API round trips incl. LAZ 1.4 formats 6–8, validation and 404 paths, CORS

Frontend tests decode real LAS/LAZ fixtures written by laspy (`frontend/src/lib/testing/fixtures/make_fixtures.py`), so compressed LAS 1.4 data is exercised end to end, and the backend is mocked. `las.test.js` runs in the Node environment because the laz-rs WASM loader needs Node's fetch and Response.

7. Known limitations

- Ground extraction is unavailable on GitHub Pages; it needs the backend hosted elsewhere (with `VITE_API_BASE` and `CORS_ORIGINS` set) or ported to run in the browser.
- Files are loaded fully into browser memory, so the practical limit is about 1.5 GiB. Larger clouds need tiling or conversion (e.g. to Cesium 3D Tiles) for streaming.
- Placement is an approximation, not a georeference; the file's CRS is not read.
- Decoding runs on the main thread, so the page is unresponsive while a large file is parsed.
- Only position and RGB are drawn: intensity, classification, NIR and extra bytes are not visualised.
- Backend jobs are kept in memory and run in threads, so a single backend instance is assumed.